CS1010 Programming Methodology

Lecture Notes

Seojin

Contents

Admin matters: . . . 4

L0: Introduction **4**

0.0 Programming . . . 4
0.1 Basic Program . . . 5
0.2 Variables and data storage . . . 5
0.2.1 Variables . . . 5
0.2.2 Format Specifiers . . . 9
0.3 Arithmetic and Assignment Operators . . . 10
0.3.1 Arithmetic Operators . . . 10
0.3.2 Unary operators . . . 10
0.3.3 Comparison Operators . . . 11
0.3.4 Logic Operators . . . 11
0.3.5 Assignment Operators . . . 11
0.3.6 Integer Division . . . 11
0.3.7 Division with `double` . . . 11
0.3.8 Operator Precedence . . . 12
0.4 Ternary Operators . . . 12
Common condition tests . . . 12
0.5 Undefined Behaviour . . . 13
Example: Division by zero . . . 13
Integer Overflow . . . 14
Key takeaway . . . 14
0.X Lab . . . 14

L1: Conditionals and Functions **15**

1.1 Abstraction I: Functions . . . 15
1.2 Conditional Statements . . . 16
1.2.1 If/else constructs . . . 16
1.2.2 Comparison and logical operators . . . 16
1.2.3 Boolean operators . . . 17
1.X Lab . . . 17

L2: Abstractions, Structs, and the Stack **18**
2.1 Abstraction II: Functions . . . 18
2.1.1 Abstraction of Behavior: Functions . . . 18
2.1.2 Abstraction of Data: Structs . . . 18
2.2 The Stack and Block Scoping . . . 19
2.2.1 Stack Frames . . . 19
2.2.2 Local variables . . . 19
2.X Lab . . . 20

L3: Fixed Size arrays and Recursion **22**
3.1 Arrays . . . 22
3.1.1 Array Declaration . . . 22
3.1.2 Initializing elements . . . 23
3.1.3 Accessing an Array (writing) . . . 23
3.1.4 Copying Semantics . . . 23
3.1.5 Arrays as arguments to functions . . . 24
3.1.6 Length of arrays . . . 24
Setting Values inside Functions . . . 24
Returning from functions . . . 25
3.2 Arrays and Structs . . . 25
Zeroing out . . . 25
UB . . . 25
Variable-length arrays . . . 25
3.3 Multidimensional Arrays . . . 26
3.3.1 Array of arrays . . . 26
3.4 Recursion . . . 26
3.4.1 Why Recursion . . . 26
3.4.2 Features of recursion . . . 26
3.4.3 Example: Printing an array in reverse . . . 27
Example: Reverse an array . . . 27
Example: Sort an array . . . 28

3.X Lab **28**

L4: Loops **28**
4.1 `while` Loops and `for` Loops . . . 28
Why loops? . . . 28
`while` loops . . . 28
`for` loops . . . 29
4.2 Loop control with `break`, `continue`, `return` . . . 30
`break` . . . 30

`continue` . . . 30
`return` . . . 30
4.3 Array recursion . . . 30
4.4 Examples . . . 31
Initialize every array element . . . 31
Nested loops: print a triangle . . . 31
A method for planning loops . . . 31

L5: Pointers and Strings **31**
5.1 Pointers . . . 31
memory . . . 31
Pointers. . . . 32
Why pointers? . . . 32
5.2 Address-of (&) and Dereferencing (*), nullptr, pointers to pointers . . . 32
5.3 Functions, structs, arrays, and pointer arithmetic . . . 32
5.4 Strings: Null termination, character arrays and string literals . . . 32
5.5 Printing strings, const and string library functions . . . 32
5.6 Pointer Provenance Framework: reasoning about pointers . . . 32
5.X Lab . . . 32

Admin matters:

Instructors:

- Dr. Eldon Chung
- Dr. Sriram Sami

Schedule:

Activity	Time	Venue
Lecture	Monday 4pm-6pm	I3-LT38
Tutorial	Wednesday 9am-10am	COM1-0120
Lab	Thursday 4pm-6pm	COM1-0120

Exams:

Activity	Date	Time
Practical Exam	Tue, 15 Sep	6pm-9pm
Midterm	Mon, 5 Oct	4pm-6pm
Practical Exam	Tue, 13 Oct	6pm-9pm
Practical Exam	Tue, 10 Nov	6pm-9pm
Final	Wed, 25 Nov	9am-11am

More info: CS1010 website

L0: Introduction

0.0 Programming

Program
: is a set of instructions that a computer runs.

Programming
: is writing code that is used to create a program.

More specifically,

1. The **CPU** (Central Processing Unit) reads information from the computer memory.
2. The CPU then writes information to the *Program Code*. The program code that we make is stored in the memory.
3. The Program stores data (Program data) for intermediate steps. These are also stored in memory.

- We write in a higher-level programming language like C to create instructions that the CPU has to execute. A **compiler** (e.g. `clang` or `gcc`) takes the C code and produces assembly code.
- We use high-level language and compilers so that our code is *portable*; we leave lower level details to be handled by the compiler.
- We also leave hardware level optimisations to be handled by compilers.

C
: is a *compiled* programming language.

Compiler
: has to take C code written to produce assembly instructions that the CPU will execute.

Pros	Cons
lots of control: imperative, low level	More work required by programmer
Simple language	Many footguns
little to no overhead	

Nevertheless, C is:

- Low-level enough for hardware (firmware)
- Widely supported across embedded systems
- Pervasive in system code
- Good starter language.

0.1 Basic Program

Anatomy:

```c
#include <stdio.h>
int main(void) {
    printf("Hello world!\n");
    return 0;
}
```

1. `#include <stdio.h>` makes the C "standard input/output" header file available to this program.
2. `int main(void) {` declares the main function, and code **starts** executing from within `main`. The `void` means that the function does not take any input arguments. The opening brace `{` means the start of the code within `main`.
3. `printf("Hello world!\n");` Creates a string `"Hello world!"` and a new line character `\n` and passes the argument to `printf`, which is sent to the standard output of the program
4. Semicolon `;` indicates the end of the statement.
5. Closing brace `}` ends the code within `main`.

Procedure:

1. Create file with extension .c, (*e.g.* `hello.c`)
2. Write C code inside.
3. Compile C code to an executable program\

```
clang -o hello hello.c
```

4. Run if no errors.

```
./hello
```

0.2 Variables and data storage

0.2.1 Variables

Variables:

Type name(s)	Min value	Max value	Size (bytes)
`char`	-128	+127	1 (8 bits)
`unsigned char`	0	+255	1 (8 bits)

Type name(s)	Min value	Max value	Size (bytes)
`int`	-32768 (-2^{15})	+32767 ($2^{15}-1$)	2
`unsigned int`	0	+65535 ($2^{16}-1$)	2
`long`	-2147483648 (2^{31})	+2147483647 ($2^{31}-1$)	4
`unsigned long`	0	+4294967295 ($2^{32}-1$)	4
`unsigned long long`	0	$2^{64}-1$	8
`float`	6-7 d.p.	6-7 d.p.	4
`double`	15-17 d.p.	15-17 d.p.	8

When to use which type

- `char` for ASCII characters
- `int` for default integral types
- `unsigned int` if integer > 0
- `float` if you want to save memory and use decimals.
- `double` if your values might be more precise decimal. Use double and instead of float, unless with good reason.

Initialization You can also initialize a variable without declaring its value.

```
int a;
int b;
a = 372;
b = a - 200;
```

Overflow If you have an `unsigned int` number at 255, and you increment it, you'll get 256 in return. As expected. If you have an `unsigned char` number at 255, and you increment it, you'll get 0 in return. It resets starting from the initial possible value.

If you have a `unsigned char` number at 255 and you add 10 to it, you'll get the number 9:

```
#include <stdio.h>

int main(void) {
    unsigned char j = 255;
    j = j + 10;
    printf("%u", j); /* 9 */
}
```

If you don't have a signed value, the behavior is undefined. It will basically give you a huge number which can vary, like in this case:

```
#include <stdio.h>

int main(void) {
    char j = 127;
    j = j + 10;
    printf("%u", j); /* 4294967177 */
}
```

In other words, C does not protect you from going over the limits of a type. You need to take care of this yourself. However, if you initialize a variable with a wrong value, you will likely receive a warning:

```
#include <stdio.h>

int main(void) {
    char j = 1000;
    char i;
    i = 9999;
```

```
    // This will give warning.
}
```

However, incrementing the number will not give a warning and overflow.

```
#include <stdio.h>

int main(void) {
    char j = 0;
    j += 1000;
}
```

ASCII table:

Dec	Symbol	Dec	Symbol	Dec	Symbol	Dec	Symbol	Dec	Symbol
48	0	63	?	78	N	93	]	108	l
49	1	64	@	79	O	94	^	109	m
50	2	65	A	80	P	95	_	110	n

Dec	Symbol	Dec	Symbol	Dec	Symbol	Dec	Symbol	Dec	Symbol
51	3	66	B	81	Q	96	| 111 | o | | 52 | 4 | 67 | C | 82 | R | 97 | a | 112 | p | | 53 | 5 | 68 | D | 83 | S | 98 | b | 113 | q | | 54 | 6 | 69 | E | 84 | T | 99 | c | 114 | r | | 55 | 7 | 70 | F | 85 | U | 100 | d | 115 | s | | 56 | 8 | 71 | G | 86 | V | 101 | e | 116 | t | | 57 | 9 | 72 | H | 87 | W | 102 | f | 117 | u | | 58 | : | 73 | I | 88 | X | 103 | g | 118 | v | | 59 | ; | 74 | J | 89 | Y | 104 | h | 119 | w | | 60 | < | 75 | K | 90 | Z | 105 | i | 120 |	106	j

Dec	Symbol	Dec	Symbol	Dec	Symbol	Dec	Symbol	Dec	Symbol
62	>	77	M	92	\	107	k	122	z

It is possible to find the size of a variable.

sizeof()

```
#include <stdio.h>
int main(void) {
    int myInt;
    float myFloat;
    double myDouble;
    char myChar;
    printf("%d\n", sizeof(myInt));
    printf("%d\n", sizeof(myFloat));
    printf("%d\n", sizeof(myDouble));
    printf("%d\n", sizeof(myChar));
    return 0;
}
```

output:

```
4
4
8
1
```

0.2.2 Format Specifiers

Format specifiers

Specifier	Output	Example
`d` or `i`	Signed decimal integer	`392`
`u`	Unsigned decimal integer	`7235`
`o`	Unsigned octal	`610`
`x`	Unsigned hexadecimal integer	`7fa`
`X`	Unsigned hexadecimal integer (uppercase)	`7FA`
`f`	Decimal floating point, lowercase	`392.65`
`F`	Decimal floating point, uppercase	`392.65`
`e`	Scientific notation (mantissa/exponent), lowercase	`3.9265e+2`
`E`	Scientific notation (mantissa/exponent), uppercase	`3.9265E+2`
`g`	Shortest representation: `%e` or `%f`	`392.65`
`G`	Shortest representation: `%E` or `%F`	`392.65`
`a`	Hexadecimal floating point, lowercase	`-0xc.90fep-2`
`A`	Hexadecimal floating point, uppercase	`-0XC.90FEP-2`
`c`	Character	`a`
`s`	String of characters	`sample`
`p`	Pointer address	`b8000000`
`n`	Nothing printed. Argument must be a pointer to a signed `int`; the number of characters written so far is stored there.	—
`%`	A literal `%` character	`%`

Below is an example:

```
#include <stdio.h>
```

```c
int main(void) {
    int sixtyseven = 67;
    double pi = 3.1415926535897;
    char ahh = 'a';
    char pain[] = "what?"; //adding [] to the end of the variable will make it a string instead of
   a char
    printf("Integer: %d\n", sixtyseven);
    printf("2dp Float: %.2f\n", pi);
    printf("6dp Float: %.6f\n", pi);
    printf("Char: %c\n", ahh);
    printf("String: %s\n", pain);

    return 0;
}
```

Integer Truncation When you output the value of `double` with `%d`, you will get the *floor* of the value, truncating the decimal point and leaving the integer part.

```c
int x = 5;
x = x + 0.1; // Behind the scenes: (int)((double)5.0 + 0.1)
printf("%d", x);
```

0.3 Arithmetic and Assignment Operators

0.3.1 Arithmetic Operators

Arithmetic operators let us do math:

Operator	Meaning	Example
`=`	Assignment	`x=1`
`+`	Addition	`x + y`
`-`	Subtraction	`x - y`
`*`	Multiplication	`x * y`
`/`	Division	`x / y`
`%`	Modulo (remainder)	`x % y`

0.3.2 Unary operators

Unary operators only take one operand:

Operator	Name	Example
`+`	Unary plus	`+a`
`-`	Unary minus	`-a`
`++`	Increment	`a++` or `++a`
`--`	Decrement	`a--` or `--a`

Example:

```c
int a = 2;
int b;
b = a++ /* b is 2, a is 3 */
b = ++a /* b is 4, a is 4 */
b = a-- /* b is 4, a is 3 */
b = --a /* b is 2, a is 2 */
```

0.3.3 Comparison Operators

Comparison Operators

Operator	Name	Example
`==`	Equal operator	`a == b`
`!=`	Not equal operator	`a != b`
`>`	Bigger than	`a > b`
`<`	Less than	`a < b`
`>=`	Bigger than or equal to	`a >= b`
`<=`	Less than or equal to	`a <= b`

0.3.4 Logic Operators

Logical operators: work with boolean values.

Operator	Name	example
`!`	NOT	`!a`
`&`	AND	`a && b`
`|`	OR	`a || b`

0.3.5 Assignment Operators

Assignment Operators perform arithmetic transformation and and assignment

Operator	Name	Example
`+=`	Addition assignment	`a += b`
`-=`	Subtraction assignment	`a -= b`
`*=`	Multiplication assignment	`a *= b`
`/=`	Division assignment	`a /= b`
`%=`	Modulo assignment	`a %= b`

0.3.6 Integer Division

If `x` and `y` are **integer-like types**, **`/` truncates the fractional part**.

```
printf("%d\n", 50 / 20);  // 2
printf("%d\n", 50 % 20);  // 10
```

- `50 / 20` → `2`, not `2.5`
- `50 % 20` → `10` (remainder)
- `%` only works on **integral types**; not `float`/`double`.

0.3.7 Division with `double`

If you want to preserve decimal values, make sure **at least one operand is `double`/`float`**:

```
int x = 50;

x / 20       // 2
x / 20.0     // 2.5
50.0 / 20.0  // 2.5
(double)x / 20  // 2.5
```

You can explicitly convert a value using a **cast**:

```
(double)x
```

0.3.8 Operator Precedence

- `*`, `/`, `%` have **higher priority** than `+` and `-`
- `()` has the **highest priority**
- Similar to the normal order of operations in mathematics.

```
x + 3 / x * (3 + x)
```

is evaluated according to these precedence rules.

> **Important:** `=` means **assignment**, not mathematical equality.

```
int x = 5;
x = x + 2;  // x is now 7
x += 2;     // x is now 9
```

Assignment **changes the state of the program** by changing the value stored in a variable.

0.4 Ternary Operators

The **ternary operator** provides a compact way to choose between two expressions based on a condition.

```
<expression1> ? <expression2> : <expression3>;
```

In English:

> If `<expression1>` is true, evaluate `<expression2>`; otherwise, evaluate `<expression3>`.

Example:

```
double product_cost = 888;

double discount_amt = (product_cost >= 670) ? 2 : 0;
```

Since `product_cost` is `888`:

```
888 >= 670 → true
```

Therefore:

```
discount_amt = 2
```

If `product_cost` were less than `670`:

```
discount_amt = 0
```

Common condition tests

Operator	Meaning
`>`	More than
`>=`	More than or equal to
`<`	Less than
`<=`	Less than or equal to
`==`	Equal to
`!=`	Not equal to

Another example:

```
(product_cost >= 670)
    ? printf("You've reached min-spend")
    : printf("Spend another %f cents\n", 670 - product_cost);
```

The ternary operator is useful for **simple conditions** where there are two possible outcomes.

0.5 Undefined Behaviour

Undefined Behaviour (UB) is a major recurring aspect of C.

C is defined by a document called the **C Standard**. CS1010 uses the **C23 standard**.

The standard specifies what should happen for valid C code. However, for some erroneous code, the standard **imposes no requirements**.

If a program contains **undefined behaviour**:

> The program is allowed to do anything.

It might:

- produce an unexpected result
- crash
- appear to work
- behave differently on another compiler/system
- do something else entirely

There are no guarantees.

Example: Division by zero

```
#include <stdio.h>

int main() {
    printf("%d\n", 5);
    printf("%d\n", 1 / 0);
    return 0;
}
```

The expression:

```
1 / 0
```

has **undefined behaviour**.

Do **not** assume that it will always produce a particular result.

Integer Overflow

Overflow occurs when a value exceeds the range that its type can represent.

Assuming `int` is 4 bytes (32 bits):

Unsigned integer overflow is well-defined:

```
unsigned int c = 4294967295;
c = c + 1;
```

The value **wraps around**:

```
c = 0
```

Signed integer overflow is undefined behaviour:

```
int c = 2147483647;
c = c + 1;  // Undefined Behaviour!
```

So:

Situation	Behaviour
Unsigned integer overflow	Well-defined; wraps around
Signed integer overflow	**Undefined Behaviour**

Key takeaway

UB = the C Standard gives no requirements for what happens.

Therefore, **never rely on what seems to happen when testing UB**. A program may behave differently depending on the compiler, optimisation, machine, or other circumstances.

UB is an unavoidable part of C, so learning to recognise and avoid it is important.

Compiler Warnings

The compiler can warn you about **some** problematic cases.

```
Listen to what your compiler says!
```

Compiler warnings are useful for catching potential problems before they become bugs.

0.X Lab

Goal: Practice working with variables, format specifiers, and the ternary operator.

Exercise 1 — Temperature conversion

Write a program that stores a temperature in Celsius as a `double`, converts it to Fahrenheit, and prints both values to 2 decimal places.

```
#include <stdio.h>

int main(void) {
    double celsius = 30.0;
    double fahrenheit = celsius * 9.0 / 5.0 + 32;
    printf("%.2f C = %.2f F\n", celsius, fahrenheit);
    return 0;
}
```

Exercise 2 — Even or odd, with a ternary

Read an `int` and use the ternary operator (not `if`) to print whether it is even or odd.

```c
#include <stdio.h>

int main(void) {
    int n = 7;
    printf("%s\n", (n % 2 == 0) ? "even" : "odd");
    return 0;
}
```

Things to watch out for

- Integer division silently truncates — cast to `double` if you need a fractional result (see 0.3.7).
- `%` only works on integer types.
- Always double check your format specifier matches the variable's type (e.g. `%f` for `double`/`float`, `%d` for `int`) — mismatching them is undefined behaviour (see 0.5).

L1: Conditionals and Functions

1.1 Abstraction I: Functions

Function
: is a named, reusable block of code that performs a specific task. It takes zero or more **arguments** as input, and can **return** a single value as output.

Why functions?

- **Abstraction** — the caller only needs to know *what* a function does (its interface), not *how* it does it (its implementation).
- **Reuse** — write the logic once, call it many times.
- **Readability** — breaking a program into small, well-named functions makes it easier to follow.

Anatomy of a function

```c
int add(int a, int b) {
    int sum = a + b;
    return sum;
}
```

- `int` (leftmost) is the **return type** — the type of value the function sends back.
- `add` is the function's **name**.
- `(int a, int b)` are the **parameters** — the inputs the function expects, along with their types.
- `return sum;` sends the value of `sum` back to whoever called the function. A function with return type `void` does not return a value, and can use a bare `return;` (or nothing) to end early.

Calling a function

```c
int result = add(3, 4); // result is 7
```

Function prototypes

If a function is defined *after* `main`, or in another file, the compiler needs to know its signature before it's used. A **prototype** declares the signature without the body:

```c
int add(int a, int b); // prototype
```

```c
int main(void) {
    int result = add(3, 4);
    return 0;
}

int add(int a, int b) { // definition
    return a + b;
}
```

1.2 Conditional Statements

1.2.1 If/else constructs

The `if` statement runs a block of code only when a condition is true.

```c
if (condition) {
    // runs when condition is true
}
```

Add an `else` to handle the opposite case:

```c
if (condition) {
    // condition is true
} else {
    // condition is false
}
```

Chain multiple conditions with `else if`:

```c
if (score >= 90) {
    printf("A\n");
} else if (score >= 75) {
    printf("B\n");
} else if (score >= 50) {
    printf("C\n");
} else {
    printf("F\n");
}
```

Only the **first** branch whose condition is true runs; the rest are skipped.

Always use braces

As seen in the L2 lab below, omitting `{ }` means only the *single statement* immediately after the `if`/`else` belongs to it — everything after that runs unconditionally regardless of the condition. Always wrap the body in `{ }`, even for one-line bodies, to avoid this class of bug.

1.2.2 Comparison and logical operators

Recall the comparison operators (`==`, `!=`, `<`, `>`, `<=`, `>=`) and logical operators (`&&`, `||`, `!`) from 0.3. Inside an `if`, these are combined to build more complex conditions.

Truth tables

`a`	`b`	`a && b`	`a || b`
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	1

a	!a
0	1
1	0

Short-circuit evaluation

`&&` and `||` evaluate left to right and stop early once the result is already decided:

- `a && b` — if `a` is false, `b` is **never evaluated** (the whole expression is already false).
- `a || b` — if `a` is true, `b` is **never evaluated** (the whole expression is already true).

This matters when the right-hand side has side effects (e.g. a function call that prints something), as seen in the `foo() && bar()` example in the L2 lab below.

1.2.3 Boolean operators

C did not have a built-in boolean type until relatively recently. `<stdbool.h>` provides one:

```c
#include <stdbool.h>

bool is_valid = true;
```

Under the hood:

- `bool` is really a small integer type; `true` is `1` and `false` is `0`.
- Any non-zero value is treated as "true" in a condition; only `0` is "false".

```c
int x = 5;
if (x) {        // true, since x != 0
    printf("x is truthy\n");
}
```

Returning a `bool` from a function

```c
#include <stdbool.h>

bool is_even(int n) {
    return n % 2 == 0;
}
```

This is the pattern used by `foo()` and `bar()` in the L2 lab below — functions that return `bool` and are combined with `&&`/`||` inside an `if`.

1.X Lab

Goal: Practice writing your own functions that use conditionals.

Exercise — Grade classifier

Write a function `char grade(int score)` that returns `'A'`, `'B'`, `'C'`, or `'F'` based on the boundaries used in 1.2.1, then call it from `main` and print the result.

```c
#include <stdio.h>

char grade(int score) {
    if (score >= 90) {
        return 'A';
    } else if (score >= 75) {
        return 'B';
```

```
    } else if (score >= 50) {
        return 'C';
    } else {
        return 'F';
    }
}

int main(void) {
    int score = 82;
    printf("Grade: %c\n", grade(score));
    return 0;
}
```

Stretch goal

Write `bool is_leap_year(int year)` using the rule: divisible by 4, except centuries, unless also divisible by 400. Combine `&&`, `||`, and `!` from 1.2.2.

L2: Abstractions, Structs, and the Stack

2.1 Abstraction II: Functions

2.1.1 Abstraction of Behavior: Functions

Section 1.1 introduced functions as reusable blocks of code. Here we look at *why* wrapping behaviour in a function is a form of **abstraction**.

When we call a function, we only need to know:

1. Its **name**
2. Its **parameters** (types and meaning)
3. Its **return type**

We don't need to know *how* it's implemented internally — that's hidden inside the function body. This is exactly what happens with `read_point()` and `print_movement()` in the lab below: `main` doesn't need to know how a point is read or how movement is computed, only that these functions exist and what they do.

Pass-by-value

In C, arguments are passed **by value** — the function receives a *copy* of the argument. Modifying a parameter inside a function does not affect the caller's variable.

```
void increment(int x) {
    x = x + 1; // only changes the local copy
}

int main(void) {
    int n = 5;
    increment(n);
    printf("%d\n", n); // still 5
}
```

2.1.2 Abstraction of Data: Structs

Just as functions abstract *behaviour*, **structs** abstract *data* — they let us group related variables into a single named type.

```
typedef struct {
    int x;
```

```
    int y;
} Point;
```

- `struct { ... }` defines a new structure with the listed fields (**members**).
- `typedef ... Point;` gives the struct the alias `Point`, so we can write `Point p;` instead of `struct { ... } p;` every time.

Creating and accessing a struct

```
Point p;
p.x = 3;
p.y = 4;
printf("(%d, %d)\n", p.x, p.y); // (3, 4)
```

The `.` (dot) operator accesses a member of a struct.

Why this is abstraction

Without a struct, a point would be two separate, unrelated `int` variables (`px`, `py`), and a function relating two points would need four separate parameters. With `Point`, related data travels together as one value — as seen in the lab below, where `print_movement(Point start, Point end)` takes just two arguments instead of four.

2.2 The Stack and Block Scoping

2.2.1 Stack Frames

Every time a function is called, the program sets aside a small block of memory called a **stack frame** to hold that call's local variables, parameters, and return address.

- When a function is **called**, a new frame is **pushed** onto the **call stack**.
- When the function **returns**, its frame is **popped** off the stack, and its memory is reclaimed.
- Each call gets its **own** frame — calling the same function twice (e.g. `read_point()` for `p` then for `q` in the lab below) creates two separate frames, each with its own copy of `pt`.

```
call main()
  -> call read_point()    [frame for read_point pushed]
  <- return                [frame for read_point popped]
  -> call read_point()    [a *new* frame pushed]
  <- return                [popped again]
  -> call print_movement() [frame pushed]
  <- return                [popped]
```

This is also why a function cannot "see" another function's local variables — they live in different frames.

2.2.2 Local variables

A **local variable** is declared inside a function or block, and only exists — is only *in scope* — within that block.

```
void foo(void) {
    int a = 1; // local to foo
}

void bar(void) {
    int a = 2; // a different `a`, local to bar
}
```

`foo`'s `a` and `bar`'s `a` are unrelated; each lives in its own stack frame.

Block scoping

Any `{ }` introduces a new scope, not just function bodies — including `if`, `else`, and loop bodies:

```
int main(void) {
    int x = 10;
    if (x > 5) {
        int y = 20;     // y only exists inside this block
        printf("%d\n", y);
    }
    // y is not visible here -- out of scope
    return 0;
}
```

Lifetime vs. scope

- **Scope** — where in the code a variable's name is visible.
- **Lifetime** — how long the variable's memory actually exists (tied to its stack frame, from when the block is entered to when it's exited).

2.X Lab

More If-Else, Abstraction, Structs

Part 1a:

Does the code below compile? What is its output?

```
#include <stdbool.h>
#include <stdio.h>

bool foo(){
    printf("foo called returning true\n");
    return true;
}

bool bar(){
    printf("bar called returning false\n");
    return false;
}

int main(){
    if(foo() && bar()){
        printf("line A reached\n");
    }

    printf("BOOP\n");

    if(foo() || bar()){
        printf("line B reached\n");
    }
}
```

Output: **Only one line output**

- If you omit the curly braces `{ }` in your if statement, only the line **below** the `if` statement is used as its output.
-

Consider the following code:

```c
#include "input.h"
#include <stdbool.h>
#include <stdio.h>

typedef struct {
    int x;
    int y;
} Point;

int main() {
    Point p;
    p.x = read_int();
    p.y = read_int();

    Point q;
    q.x = read_int();
    q.y = read_int();

    int dx = q.x - p.x;
    int dy = q.y - p.y;

    bool moving_right = dx > 0;
    bool moving_up = dy > 0;
    bool on_same_x = dx == 0;
    bool on_same_y = dy == 0;

    if (on_same_x && on_same_y) {
        printf("same point\n");
    } else if (on_same_x) {
        printf("vertical movement\n");
    } else if (on_same_y) {
        printf("horizontal movement\n");
    } else if (moving_right && moving_up) {
        printf("northeast\n");
    } else if (!moving_right && moving_up) {
        printf("northwest\n");
    } else if (moving_right && !moving_up) {
        printf("southeast\n");
    } else {
        printf("southwest\n");
    }
    return 0;
}
```

We can simplify this to:

```c
#include "input.h"
#include <stdbool.h>
#include <stdio.h>

typedef struct {
    int x;
    int y;
} Point;

Point read_point() {
    Point pt;
    pt.x = read_int();
    pt.y = read_int();
    return pt;
}

void print_movement(Point start, Point end) {
    int dx = end.x - start.x;
    int dy = end.y - start.y;
```

```
    bool moving_right = dx > 0;
    bool moving_up = dy > 0;
    bool on_same_x = dx == 0;
    bool on_same_y = dy == 0;

    if (on_same_x && on_same_y) {
        printf("same point\n");
    } else if (on_same_x) {
        printf("vertical movement\n");
    } else if (on_same_y) {
        printf("horizontal movement\n");
    } else if (moving_right && moving_up) {
        printf("northeast\n");
    } else if (!moving_right && moving_up) {
        printf("northwest\n");
    } else if (moving_right && !moving_up) {
        printf("southeast\n");
    } else {
        printf("southwest\n");
    }
}

int main() {
    Point p = read_point();
    Point q = read_point();
    print_movement(p, q);
    return 0;
}
```

L3: Fixed Size arrays and Recursion

3.1 Arrays

3.1.1 Array Declaration

Let's say we want to declare an array that stores 5 elements of type `double` that we want to call "costs".

An **array** is declared as such: `<type> <array name>[<no. of elements>]`

```
double costs[5];
```

> **Important:** Arrays have a fixed type. You cannot add multiple different types to an array unlike Python.

Arrays are stored on the stack.

```
main

costs = {?,?,?,?,?}
```

An array will have all of its elements stored contiguously to each other in memory. * This has implications for both performance and security in the far future.

```
double costs[5] = {8.88, 10.22, 9.88, 22.22, 44.44};
        // initializes the array with 5 values
```

It is stored in the stack as such:

```
stack:
|8.88|10.22|9.88|22.22|44.44|
```

3.1.2 Initializing elements

However, what if we initialize only 3 elements out of 5?

```
double costs[5]= {8.88,10.22,9.88};
// the remaining elements will be initialized to 0.0 by default.


double zeroarray[5]={0};
//^^this is a very good way to initialize an entire array to 0!
```

What if we initialize more than 5 elements?

```
double costs[5] = {8.88, 10.22, 9.88, 22.22, 44.44, 1.0};
```

If we have more values than the specified size, what happens is up to the compiler (technically not undefined behavior) [we don't recommend doing this, there's no good reason to at this stage]

What if we initialize it over 2 lines?

```
double costs[5];
costs = {8.88, 10.22, 9.88, 22.22, 44.44, 1.0};
```

> This does **NOT** work and throws a `compile error`. Unlike scalar variables, arrays cannot be assigned new values as a whole.

3.1.3 Accessing an Array (writing)

We use the index.

```
double costs[5];

costs[0] = 8.88;
// | 8.88 | ? | ? | ? | ? |

costs[4] = 44.44;
// | 8.88 | ? | ? | ? | 44.44 |
```

3.1.4 Copying Semantics

```
int main(void) {
    double costs[5] = {8.88, 10.22, 9.88, 22.22, 44.44};
    double costs_2[] = costs; // error!
}
```

> This does **NOT** work and will throw a *compile error*! `error: array initializer must be an initializer list.`

Arrays (unlike numerical types and structs) as a whole are **not copyable** via assignment operators.

3.1.5 Arrays as arguments to functions

```c
#include <stdio.h>
#include <stddef.h>

void print_doublearr_index(double arr[], size_t index) {
    printf("%f\n", arr[index]);
}

int main(void) {
    double costs[5] = {8.88, 10.22, 9.88, 22.22, 44.44};
    print_doublearr_index(costs, 3);
}
```

`size_t`: unsigned type (typically `unsigned long`)

- Guaranteed to be large enough to index any array
- Intent is clear: use by default for array indices, unless we need negative values
- `size_t` is defined in `<stddef.h>`

Array indices can technically be any integral type (`char`, `int`, `long` ...)

3.1.6 Length of arrays

```c
void print_last_element (double arr[]) {
    printf("%f\n", arr[?])
}

int main(void) {
    double costs[5] = {8.88, 10.22, 9.88, 22.22, 44.44};
    print_last_element(costs);
}
```

In **C**, we do **NOT** have `length/len` operators to measure the length of an array.

Instead, we use `size_t` for now:

```c
void print_last_element(double arr[], size_t length) {
    printf("%f\n", arr[length - 1]);
}

int main(void) {
    double costs[5] = {8.88, 10.22, 9.88, 22.22, 44.44};
    print_last_element(costs, 5);
}
```

Setting Values inside Functions

```c
int main(void) {
    double costs[5] = {8.88, 10.22, 9.88, 22.22, 44.44};
    set_doublearr_index(costs, 3, 5.5);
    printf("%f\n", costs[3]);
}
void set_doublearr_index(double arr[], size_t index, double val) {
    arr[index] = val;
}
    //output is... 5.5.
```

This sets the index 3 (the 4th) element of costs to 5.5. But how is that possible? This is because arrays are pointers.

Returning from functions

For now, do not return arrays declared within functions.

```
// DONT DO THIS
??? foo() {
    double costs[5] ={8.88, 10.22, 9.88, 22.22, 44.44, 1.0};
    return costs;
} // will return UB
```

3.2 Arrays and Structs

Zeroing out

```
#include <stdio.h>

typedef struct Position {
    int x;
    int y;
} Position;

int main(void){
    Position positions[2] = {0};
    printf("%d %d\n", positions[0].x, positions[0].y);
} // This will zero out all members of all structs.
```

UB

```
int main(void) {
    double costs[1] = {8.88};
    printf("%f\n", costs[1]);
}

// UB [1]


int main(void) {
    double costs[1];
    printf("%f\n", costs[-1]);
}

// UB [2]

??? foo() {
    double costs[1] = {8.88};
    return costs;
}

// UB [3]

int main() {
    double costs[1];
    printf("%f\n", costs[0]);
}
```

Variable-length arrays

Can we declare the size of an array as a variable? Yes.

```
double costs[var];
```

However, there are much better ways to do this (discussed later). We do not allow VLAs because of underlying issues. As a matter of fact, VLA was removed from Linux operating system.

3.3 Multidimensional Arrays

3.3.1 Array of arrays

```
int main(void) {
    int a[2][3] = {{0,1,2},{3,4,5}};
    printf("%d\n", a[1][0]);
}
```

Then a is stored as:

$$a = \begin{bmatrix} 0 & 1 & 2 \\ 3 & 4 & 5 \end{bmatrix}$$

We are storing an array of arrays.

Example: tic-tac-toe

```
int grid[3][3] = {{1,2,3},{4,5,6},{7,8,9}};
```

$$\text{grid} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Arrays can have more dimensions!

Only the first dimension size can be omitted.

3.4 Recursion

3.4.1 Why Recursion

You **cannot call yourself a programmer if you don't know how to recurse**. You will be locking yourself away from future concepts if you don't learn this. This includes:

- Binary search trees
- Fast/Efficient sorting algorithms
- A lot of efficient algorithms. The fast Fourier transform (FFT) is an example

3.4.2 Features of recursion

- **Base Case:**
  - The simplest possible case that we have to solve manually.
- **Recursive Case:**
  - In this case we usually have to "steer" the general case into the smaller case somehow.
  - We will rely on Cheems to solve the smaller version of the problem for us.
  - We will use whatever Cheems gives us + a little bit more work to solve the problems in general.

```
<return_type> foo(<param(s) here>){
    if(<base case>){
    return; // or return value;
}

// handle recursive case here
// any computation before recursive call(s)

foo(<with smaller case>);
    // any computation after recursive call(s)
}
```

3.4.3 Example: Printing an array in reverse

Logic:

1. The base case is when the length of the array is 0, so we do nothing.
2. We have an entire array of n elements.
3.

- The entire array of n elements
  - The last element will be our base case.
  - Cheems will handle the rest with: `print_arr_reverse(arr, len-1)`
- We should do something only with the last element. We print the last element before Cheems.

```
void print_arr_reverse(int arr[], size_t len){
    if (len == 0) {
        return; // array has nothing, we do nothing.
    }

    printf("%d\n", arr[len-1]);

    print_arr_reverse(arr, len-1); // cheems!
}
```

Example: Reverse an array

Input: An array of length `len`. Output: Reverse the elements of `arr`.

We need a helper function here: The function will need:

- take in an array
- Swap the first and last elements

```
void reverse_arr_helper(int arr[], size_t start, size_t end) {
    if (start >= end) {
        return;
    }
    reverse_arr_helper(arr, start+1, end-1);
    int temp = arr[start];
    arr[start] = arr[end];
    arr[end] = temp;
}

void reverse_arr(int arr[], size_t len) {
    if (len==0) {return;}
    reverse_arr_helper(arr,0,len-1);
}
```

Example: Sort an array

Let's say Cheems has already sorted the first $n-1$ elements for us. What should we do with the last element? > We figure out where to put it.

```
void shift_left(int arr[], size_t len, size_t count) {
    ...
}

void count_larger_than(int arr[], size_t len, size_t val) {
    ...
}

void sort_arr(int arr[], size_t len) {
    if(len==0){
        return;
    }
    size_t count = count_larger_than(arr, len-1, arr[len-1]);
    shift_left(arr, len, count);
}
```

3.X Lab

- When declaring `arrays[][]`, you need to at least declare the no. of columns `[][cols]`.
- `void` will not affect the variable inside `main` without `return`.
- However, `void` calls on the array directly and can redefine the array elements.

L4: Loops

4.1 `while` Loops and `for` Loops

Why loops?

Some problems can be solved using recursion, but recursion can use a lot of stack space. Consider printing every element of an array:

```
void print_arr_from(double arr[], size_t length, size_t i) {
    if (i == length) {
        return;
    }
    printf("%f\n", arr[i]);
    print_arr_from(arr, length, i + 1);
}

void print_arr(double arr[], size_t length) {
    print_arr_from(arr, length, 0);
}
```

Each recursive call uses a stack frame and waits for the next call to finish. Printing an array of length n creates roughly n nested calls, so a sufficiently large array can exhaust the stack and cause a **stack overflow**. C does not guarantee tail-call optimization to avoid this.

Loops let us repeat code without making a new function call for every repetition.

`while` loops

```
while (expression) {
```

```
    // loop body
}
```

The expression is checked first. If it is true, the loop body runs. At the end of the body, execution returns to the condition and checks it again. If the expression is false, the loop body is skipped or the loop ends.

For example, print every element of an array:

```
void print_arr(double arr[], size_t length) {
    size_t i = 0;

    while (i < length) {
        printf("%f\n", arr[i]);
        i += 1;
    }
}
```

Here, `i` is the **index** (or iterator). It starts at zero, increases after each iteration, and the loop ends when `i < length` becomes false.

If the loop body never changes anything that can make the condition false, the loop may run forever. For example, leaving out `i += 1` would repeatedly print `arr[0]`.

Array indices go from `0` to `length - 1`. Using `i <= length` would eventually access `arr[length]`, which is outside the array and causes undefined behavior.

For reverse iteration with `size_t`, avoid a condition such as `i >= 0`: `size_t` is unsigned, so that condition is always true, even after the value wraps around. One way to print in reverse is:

```
void print_arr_reverse(double arr[], size_t length) {
    for (size_t i = 0; i < length; i += 1) {
        printf("%f\n", arr[(length - 1) - i]);
    }
}
```

`for` loops

A `for` loop puts initialization, condition, and iteration in one place:

```
for (expression1; expression2; expression3) {
    // loop body
}
```

- `expression1` runs once before the loop starts. It often declares and initializes an index.
- `expression2` is the condition. The body runs only when it is true.
- `expression3` runs after each execution of the body. It often updates the index.

The `while` and `for` versions below do the same thing:

```
size_t i = 0;
while (i < length) {
    printf("%f\n", arr[i]);
    i += 1;
}
```

```
for (size_t i = 0; i < length; i += 1) {
    printf("%f\n", arr[i]);
}
```

Variables declared in a `for` loop's initialization are scoped to the loop, so `i` cannot be used after the loop. A variable declared before a `while` loop remains accessible after it.

4.2 Loop control with `break`, `continue`, `return`

`break`

`break` immediately exits the **innermost loop** containing it. Execution continues after that loop.

```
for (size_t i = 0; i < 5; ++i) {
    printf("%zu\n", i);
    if (i == 1) {
        break;
    }
}
printf("Loop ended!\n");
```

This prints `0`, `1`, and `Loop ended!`. In nested loops, `break` exits only the inner loop.

`continue`

`continue` skips the rest of the current loop body and moves to the next iteration. In a `for` loop, the iteration expression runs before the condition is checked again.

```
for (size_t i = 0; i < 5; ++i) {
    if (i == 1) {
        continue;
    }
    printf("%zu\n", i);
}
```

This prints `0`, `2`, `3`, and `4`.

`return`

`return` immediately exits the **entire function**, including any loops inside it. For example, this function returns as soon as it finds a `1`:

```
bool has_one_inside_array(int arr[], size_t length) {
    for (size_t i = 0; i < length; ++i) {
        if (arr[i] == 1) {
            return true;
        }
    }
    return false;
}
```

4.3 Array recursion

Array elements are commonly processed by iterating over their indices:

```
for (size_t i = 0; i < length; ++i) {
    // process arr[i]
}
```

This is useful for tasks such as printing every element, initializing an array, and calculating a sum. The index starts at `0`, and the loop condition `i < length` ensures the last index accessed is `length - 1`.

4.4 Examples

Initialize every array element

```
void init_array(double arr[], size_t length, double val) {
    for (size_t i = 0; i < length; ++i) {
        arr[i] = val;
    }
}
```

`++i`, `i++`, and `i += 1` all increase `i` by one here.

Nested loops: print a triangle

A loop can be placed inside another loop. The inner loop runs to completion during each outer loop iteration.

```
void print_triangle(size_t height) {
    for (size_t i = 0; i < height; ++i) {
        for (size_t j = 0; j < i + 1; ++j) {
            printf("X ");
        }
        printf("\n");
    }
}
```

The outer loop controls the row number. The inner loop prints `i + 1` Xs on that row. Each loop's index is scoped to its own `for` loop.

A method for planning loops

1. Decide what condition must be true for the loop to continue.
2. If the loop needs an iterator, choose its type and initialize it.
3. Decide how to update the iterator after each iteration.

L5: Pointers and Strings

5.1 Pointers

memory

Before we talk about pointers, we need to first about memory. Memory has two Features:

- **address** is the location of where data is stored
- **value** of the memory is the data stored at the location

For example:

```
int x = 4;
int *p = &x;
```

Then the memory would look like so:

Address	Value
'0x1000'	'0x4'
'0x1004'	'0x1000'

Address	Value
'0x1008'	
'0x100C'	
'0x1010'	
'0x1014'	
'0x1018'	
'0x101C'	

Pointers.

A pointer is a value that happens to be a sign that points to another value's address. We define integer `int x = 4`. In english, "integer named `x` is set to 4." And then we define `int *p = &x` In english, "integer pointer named `p` is set to the *address* of `x`" The asterisk `*` stands for pointer, while the ampersand `&` stands for address.

```
int *p = &x;
int y = *p;
```

when an asterisk is used without the type, it becomes a pointer **dereference**. In english, "integer named `y` is set to *the thing pointed to* by `p`."

```
int x =1;
int *ptr = &x;
printf("%d\n",*ptr);
// 1
```

`int *ptr = &x` - type pointer to an integer. `&` means address. `*ptr = 1;` - dereferencing a pointer to its address

Why pointers?

We break down functions in C based on the actions they perform. However, they are sometimes out of scope that we want. To get around this, we pass the struct using a pointer so that the pointer to the struct is in scope of the functions we want. This reduces the space by not copying.

Necessary for memory allocation.

5.2 Address-of (&) and Dereferencing (*), nullptr, pointers to pointers

5.3 Functions, structs, arrays, and pointer arithmetic

5.4 Strings: Null termination, character arrays and string literals

5.5 Printing strings, const and string library functions

5.6 Pointer Provenance Framework: reasoning about pointers

5.X Lab